CLASS 04

Transforming Variables & Causality

GOVT 10: Quantitative Political Analysis

> This chapter has two halves. First we learn to transform and create variables: recoding, conditional logic, and building the measures our questions actually require. Then we turn to causality and the three requirements that separate "these move together" from "this causes that."

1 Transforming and Creating Variables

In the first two weeks of this course, we learned how to read political data into R, inspect it with `glimpse()`, filter rows with `filter()`, sort with `arrange()`, and summarize with `group_by()` and `summarise()`. Those tools work beautifully when the columns we need already exist. They almost never do.

Consider a state legislative dataset that records each member's birth year, party, district number, and roll-call votes. A useful research question might ask whether younger legislators of either party are more likely to support a climate bill. To answer it, we need an age variable rather than a birth year, an age category that lets us group "younger" members together, an indicator for the climate vote that distinguishes a yes from an abstention, and perhaps a flag for districts that flipped party control in the last election. None of those variables are in the raw file. Every one of them must be built.

This is the everyday work of quantitative political science. A common rule of thumb is that researchers spend roughly two-thirds of their analysis time transforming variables and only the remaining third running models. Transformation is not preliminary busywork; it is where most of the analytical thinking happens. Defining "competitive district," "high turnout," or "swing voter" requires us to make decisions that shape every result that follows.

1.1 Creating Variables with mutate()

The tidyverse verb for creating and modifying columns is `mutate()`. It takes a data frame and returns a new data frame with one or more additional columns attached. The original columns remain in place. Here is the basic shape:

```
districts %>%
  mutate(
    total_votes = dem_votes + rep_votes,
    dem_share   = dem_votes / total_votes,
```

```
    margin      = dem_votes - rep_votes
  )
```

```
# A tibble: 8 x 8
  district_id state dem_votes rep_votes incumbent_party total_votes dem_share margin
        <int> <chr>     <dbl>     <dbl> <chr>                 <dbl>     <dbl>  <dbl>
1           1 PA       118000    122000 R                    240000     0.492  -4000
2           2 PA       132000    119000 D                    251000     0.526  13000
3           3 MI        95000    108000 R                    203000     0.468 -13000
4           4 MI       142000    134000 D                    276000     0.514   8000
5           5 AZ        88000     96000 R                    184000     0.478  -8000
6           6 AZ       121000    117000 D                    238000     0.508   4000
7           7 GA       105000    103000 D                    208000     0.505   2000
8           8 GA        99000    112000 R                    211000     0.469 -13000
```

Notice the pattern: a data frame piped into `mutate()`, then one new column per line, each defined as `new_name = some_expression`. Everything below this point is variation on that template.

> **What this code does:** Starting from a small dataset of eight congressional districts, we use `mutate()` to add three new columns. `total_votes` is the sum of Democratic and Republican votes. `dem_share` converts the raw count into a proportion. `margin` captures the gap between the parties, with positive numbers indicating a Democratic lead. Each calculation is performed row by row, so every district gets its own values. The original columns are untouched.

Three properties of `mutate()` are worth holding in mind. First, calculations are vectorized: every row is processed simultaneously, so we never write loops. Second, mutate operates inside the pipe, which means it composes cleanly with filter, group_by, and summarise. Third, mutate is fundamentally different from `summarise()`: mutate keeps the original number of rows and adds columns, while summarise collapses many rows into one. A useful slogan: mutate decorates a dataset, summarise digests it.

> **Mutate Versus Summarise**
>
> **mutate()** returns a dataset with the same number of rows. Use it when you want each observation to carry a new value (an age category, a competitiveness flag, a per-capita rate).
>
> **summarise()** returns one row per group. Use it when you want a single number that describes a group (a mean, a count, a proportion).
>
> A common mistake is to use `summarise()` when you wanted `mutate()`, then notice that all your other columns have disappeared. If you still need every row, reach for `mutate()`.

1.2 Logical Operators and TRUE/FALSE Values

Most variable transformations rest on a simple foundation: logical comparison. R recognizes six comparison operators, summarized in Table 3.1. Each takes two values and returns `TRUE`, `FALSE`, or `NA`.

Operator	Meaning	Example
`==`	Equal to	`party == "Democrat"`
`!=`	Not equal to	`state != "CA"`
`>`	Greater than	`age > 65`
`>=`	Greater than or equal to	`margin >= 0.05`
`<`	Less than	`turnout < 0.50`
`<=`	Less than or equal to	`years_in_office <= 2`

Table 1: Comparison Operators in R

A useful mnemonic for the easiest beginner trap is the "triad of equals." The assignment arrow `<-` makes an object *get* a value. A single `=` sets a function argument inside parentheses. The double `==` asks a question and returns `TRUE` or `FALSE`. The three look similar but do very different things, and mixing them produces some of the most common errors students see in the first month of the course.

Once we have logical values, two operators combine them. The ampersand `&` is logical AND: it returns `TRUE` only when both sides are true. The pipe `|` is logical OR: it returns `TRUE` when either side is true. These let us express compound questions such as "young Democratic voters" or "competitive Senate races."

When `&` and `|` appear in the same expression, R applies `&` before `|` – but rather than memorize precedence rules, **use parentheses to make the grouping explicit**: `(a > 3 & b < 5) | c == 7` is unambiguous, whereas `a > 3 & b < 5 | c == 7` invites bugs.

```
races %>%
  mutate(
    competitive       = abs(dem_share - 50) < 5,
    safe_dem          = dem_share >= 55,
    competitive_senate = abs(dem_share - 50) < 5 & chamber == "Senate",
    flipped           = (dem_share > 50 & incumbent_party == "R") |
                        (dem_share < 50 & incumbent_party == "D")
  )
```

```
# A tibble: 10 x 9
   race_id state chamber dem_share incumbent_party competitive safe_dem competitive_senate
     <int> <chr> <chr>       <dbl> <chr>           <lgl>       <lgl>    <lgl>
 1       1 PA    Senate       48.2 D               TRUE        FALSE    TRUE
 2       2 OH    Senate       41.5 R               FALSE       FALSE    FALSE
 3       3 AZ    House        52.8 D               TRUE        FALSE    FALSE
 4       4 GA    Senate       49.4 D               TRUE        FALSE    TRUE
 5       5 NV    House        50.9 R               TRUE        FALSE    FALSE
 6       6 WI    House        53.1 R               TRUE        FALSE    FALSE
 7       7 MI    Senate       50.2 D               TRUE        FALSE    TRUE
 8       8 TX    House        38.6 R               FALSE       FALSE    FALSE
 9       9 FL    Senate       45   R               FALSE       FALSE    FALSE
10      10 NC    House        47.8 R               TRUE        FALSE    FALSE
# i 1 more variable: flipped <lgl>
```

What this code does: For each race we create four logical (TRUE/FALSE) columns. `competitive` flags races decided by fewer than five points either way. `safe_dem` flags races the Democrat won with at least a 10-point cushion. `competitive_senate` combines two conditions with `&`, so only competitive Senate races qualify. `flipped` uses `|` to capture races where the party in power lost the seat, either direction. Logical columns are useful because R stores them as 0s and 1s under the hood, which means `sum()` counts how many are TRUE and `mean()` returns the proportion that are TRUE.

The fact that R treats `TRUE` as 1 and `FALSE` as 0 unlocks a very common shortcut. Counting competitive races becomes `sum(competitive)`. Calculating the competitive race rate becomes `mean(competitive)`. Counting competitive Senate contests becomes `sum(abs(dem_share - 50) < 5 & chamber == "Senate")`. We never need to write a loop or a counter; logical arithmetic does the work.

1.3 Binary Recoding with if_else()

When a decision has exactly two outcomes, the cleanest tool is `if_else()`. Its syntax mirrors a sentence in English: `if_else(condition, value_if_true, value_if_false)`.

```
races %>%
  mutate(
    outcome    = if_else(dem_share > 50, "Dem Win", "Rep Win"),
    chamber_tag = if_else(chamber == "Senate", "Upper Chamber", "Lower Chamber")
  ) %>%
  select(race_id, state, chamber, dem_share, outcome, chamber_tag)
```

```
# A tibble: 10 x 6
   race_id state chamber dem_share outcome chamber_tag
     <int> <chr> <chr>       <dbl> <chr>   <chr>
 1       1 PA    Senate       48.2 Rep Win Upper Chamber
 2       2 OH    Senate       41.5 Rep Win Upper Chamber
 3       3 AZ    House        52.8 Dem Win Lower Chamber
 4       4 GA    Senate       49.4 Rep Win Upper Chamber
 5       5 NV    House        50.9 Dem Win Lower Chamber
 6       6 WI    House        53.1 Dem Win Lower Chamber
 7       7 MI    Senate       50.2 Dem Win Upper Chamber
 8       8 TX    House        38.6 Rep Win Lower Chamber
 9       9 FL    Senate       45   Rep Win Upper Chamber
10      10 NC    House        47.8 Rep Win Lower Chamber
```

What this code does: `if_else()` replaces a numeric or text column with a labeled categorical one. The first call checks whether the Democratic vote share exceeded 50 and writes "Dem Win" or "Rep Win" accordingly. The second call turns the `chamber` text into a more descriptive label. Compared with base R's `ifelse()`, the tidyverse `if_else()` is strict about types, which catches subtle bugs where, for example, one branch returns text and the other returns a number.

Use `if_else()` when the answer is genuinely binary: voted or did not vote, won or lost, urban or rural, above or below a threshold. As soon as you find yourself writing nested `if_else()` calls inside other `if_else()` calls, you have outgrown this tool and should reach for `case_when()`.

What does it mean that `if_else()` is "strict"? **Both branches must return the same type.** If the TRUE branch returns text and the FALSE branch returns a number, R errors out instead of silently coercing – which catches the bug at write time rather than letting it corrupt your analysis downstream.

1.4 Multiple Categories with case_when()

`case_when()` is the workhorse for recoding into three or more categories. The pattern reads like a stack of if/then rules: each line states a condition, then a tilde ~, then the value to assign when the condition is true. R works through the rules from top to bottom and takes the first one that matches.

```
candidates %>%
  mutate(
    competitiveness = case_when(
      dem_share < 45  ~ "Safe Rep",
      dem_share < 49  ~ "Lean Rep",
      dem_share < 51  ~ "Toss-Up",
      dem_share < 55  ~ "Lean Dem",
      TRUE            ~ "Safe Dem"
    ),
    ideology_category = case_when(
      ideology_score < -0.4 ~ "Very Liberal",
      ideology_score <  0   ~ "Liberal",
      ideology_score <  0.4 ~ "Conservative",
      TRUE                  ~ "Very Conservative"
    )
  ) %>%
  select(candidate_id, dem_share, competitiveness, ideology_score, ideology_category)
```

```
# A tibble: 12 x 5
   candidate_id dem_share competitiveness ideology_score ideology_category
          <int>     <dbl> <chr>                    <dbl> <chr>
 1            1      38.5 Safe Rep                  0.62 Very Conservative
 2            2      42.1 Safe Rep                  0.45 Very Conservative
 3            3      46.3 Lean Rep                  0.28 Conservative
 4            4      48.7 Lean Rep                  0.11 Conservative
 5            5      49.6 Toss-Up                   0.05 Conservative
 6            6      50.2 Toss-Up                  -0.02 Liberal
 7            7      50.9 Toss-Up                  -0.09 Liberal
 8            8      52.4 Lean Dem                 -0.18 Liberal
 9            9      54.1 Lean Dem                 -0.31 Liberal
10           10      57.8 Safe Dem                 -0.44 Very Liberal
11           11      62.5 Safe Dem                 -0.55 Very Liberal
12           12      68   Safe Dem                 -0.71 Very Liberal
```

What this code does: For each candidate we create two categorical variables. `case_when()` evaluates each line in order and assigns the value to the right of the tilde when the condition on the left is TRUE. The first block converts the continuous Democratic vote share into a five-level competitiveness label that pollsters and forecasters use every cycle. The second block bins a DW-NOMINATE-style ideology score into four categories. The final `TRUE ~` line is a catch-all: it fires whenever none of the earlier conditions matched. Without that final line, anyone who fell outside every numeric range

would receive `NA` instead of a category, which is almost never what you want.

Four habits make `case_when()` reliable. First, **order matters**: R evaluates each line top-to-bottom and assigns the first match, so when conditions overlap, the *earliest* condition wins. Put the most specific or most restrictive condition first, the catch-all last. Second, always include the final `TRUE ~ ...` line. Third, every branch should return the same type. Mixing strings and numbers across branches will produce confusing errors. Fourth, if a respondent could plausibly match multiple conditions, write the conditions so that exactly one matches each row.

Cleaning Messy Categorical Data

Survey data often contains the same concept written many ways: "Democrat," "Dem," "Democratic," "dem," "liberal." `case_when()` combined with `%in%` or pattern matching can collapse these into clean categories.

Using %in% for set membership:

```
party_clean = case_when(
  party_raw %in% c("Democrat", "Democratic", "dem", "liberal")
    ~ "Democrat",
  party_raw %in% c("Republican", "GOP", "rep", "conservative")
    ~ "Republican",
  party_raw %in% c("Independent", "ind", "none", "other")
    ~ "Independent",
  TRUE ~ NA_character_
)
```

The operator `%in%` checks whether a value appears in a vector. It is far more readable than chaining multiple == conditions with |.

1.5 Helpful Companions: pull(), n_distinct(), across(), recode()

Four additional functions round out the transformation toolkit.

The function `pull()` extracts a single column as an ordinary vector. **Why we need it:** `summarise()` always returns a tiny tibble (a one-row, one-column data frame), even when there is only one number inside. To use that number in downstream arithmetic (comparisons, multiplications, vector operations), we have to `pull()` it out so it behaves like a plain scalar. For example, we might compute the median Democratic vote share across our races and then use that single number to flag every candidate as running ahead of or behind the field.

```
median_share <- candidates %>%
  summarise(med = median(dem_share)) %>%
  pull(med)

candidates %>%
  mutate(above_median = dem_share > median_share) %>%
  select(candidate_id, dem_share, above_median)
```

```
# A tibble: 12 x 3
```

```
   candidate_id dem_share above_median
          <int>     <dbl> <lgl>
 1            1      38.5 FALSE
 2            2      42.1 FALSE
 3            3      46.3 FALSE
 4            4      48.7 FALSE
 5            5      49.6 FALSE
 6            6      50.2 FALSE
 7            7      50.9 TRUE
 8            8      52.4 TRUE
 9            9      54.1 TRUE
10           10      57.8 TRUE
11           11      62.5 TRUE
12           12      68   TRUE
```

> **What this code does:** First we compute the median Democratic vote share and use `pull()` to extract it as a single number, stored in `median_share`. Then we use that number inside `mutate()` to create a logical flag for every candidate. Without `pull()`, `median_share` would remain a one-row, one-column tibble, which behaves differently in arithmetic.

The function `n_distinct()` counts the number of unique values in a column. It is invaluable for sanity-checking categorical variables before analysis.

```
candidates %>%
  summarise(
    n_party_labels = n_distinct(party_raw),
    total_candidates = n()
  )
```

```
# A tibble: 1 x 2
  n_party_labels total_candidates
           <int>            <int>
1              9               12
```

> **What this code does:** `n_distinct()` returns the number of unique values, while `n()` returns the total number of rows. If our cleaning is correct, a column like `party_clean` should have only three distinct values once we have collapsed all the variants. `n_distinct()` lets us check.

The function `across()` applies the same transformation to multiple columns at once. Rather than repeating ourselves, we can recode a whole block of policy-position columns in a single line.

```
votes %>%
  mutate(across(ends_with("_vote"),
                ~ if_else(.x == "Yes", 1, 0),
                .names = "{.col}_num"))
```

```
# A tibble: 6 x 7
  legislator_id health_vote climate_vote tax_vote health_vote_num climate_vote_num
          <int> <chr>       <chr>        <chr>               <dbl>             <dbl>
1             1 Yes         Yes          No                      1                 1
2             2 No          No           Yes                     0                 0
3             3 Abstain     Yes          No                      0                 1
4             4 Yes         No           Yes                     1                 0
5             5 Yes         Yes          No                      1                 1
6             6 No          Abstain      Yes                     0                 0
```

```
# i 1 more variable: tax_vote_num <dbl>
```

> **What this code does:** `across()` applies the same function to a group of columns selected by `ends_with("_vote")`. The lambda `~ if_else(.x == "Yes", 1, 0)` converts each Yes/No/Abstain value into a 1 or 0. The `.names = "{.col}_num"` argument creates new columns with the suffix `_num` rather than overwriting the originals.

Finally, `recode()` is a quick way to swap one set of values for another within a single column. It is the simplest recoding tool, well suited to small lookup tasks.

```
candidates %>%
  mutate(party_short = recode(party_raw,
                              "Democrat"     = "D",
                              "Democratic"   = "D",
                              "dem"          = "D",
                              "liberal"      = "D",
                              "Republican"   = "R",
                              "GOP"          = "R",
                              "rep"          = "R",
                              "conservative" = "R",
                              .default       = "Other")) %>%
  select(candidate_id, party_raw, party_short)
```

```
# A tibble: 12 x 3
   candidate_id party_raw   party_short
          <int> <chr>       <chr>
 1            1 rep         R
 2            2 Republican  R
 3            3 GOP         R
 4            4 Republican  R
 5            5 rep         R
 6            6 Independent Other
 7            7 ind         Other
 8            8 Democrat    D
 9            9 dem         D
10           10 Democratic  D
11           11 Democrat    D
12           12 liberal     D
```

> **What this code does:** `recode()` replaces specified values with new values. Anything not explicitly listed falls into the category specified by `.default`. For larger or more complex recoding, `case_when()` is more flexible, but `recode()` is concise for simple one-to-one swaps.

1.6 From Variables to Questions

Once we can build any variable we want, a new question arises: what should we build? The choice of categories is itself an analytic decision. Should "competitive" mean a margin under five percentage points, three points, or one point? Should "senior" begin at 60, 65, or 67? Should "high turnout" be defined relative to the district's history or relative to the national average?

These choices are not arbitrary, but they are also not forced by the data. They reflect a theory about what matters. **Theory and prior research drive the cutpoints**: the Cook Political Report's

"competitive" threshold, the Census Bureau's age brackets, and a published literature on turnout will each suggest defensible defaults. When prior work doesn't give you a clean answer, try several reasonable cutpoints and check whether your conclusions are robust to the choice – if they aren't, that fragility is itself a finding worth reporting.

2 Causality

2.1 From Description to Causation

In Chapter 2 you learned how to describe a single variable: its center, spread, and shape. In Chapter 3 you learned how to compare two variables and ask whether they are related. Those tools are powerful, but they stop short of the questions that motivate most political science. Does raising the minimum wage cost jobs? Do get-out-the-vote (GOTV) mailers actually mobilize voters? Does negative campaign advertising help or hurt the candidate who goes on the attack? Does exposure to immigrants change native-born citizens' attitudes about refugee integration?

Each of these questions is causal. Each requires us to move past the patterns we observe in data and reason about what would have happened under different conditions. The descriptive techniques from earlier chapters can tell us that two things are associated, but association is the beginning of a causal investigation, not the end. This chapter develops a framework for thinking carefully about causation and a vocabulary for spotting when a causal claim is well-supported and when it is not.

2.2 A Cautionary Example: Ice Cream and Crime

Around a decade ago, a widely shared article noted that counties with higher ice cream sales also had higher rates of violent crime. The correlation was strong and the data were real. Should city councils restrict ice cream parlors to reduce assault?

Your intuition says no, and the intuition is right. The relationship is spurious. Hot weather drives both ice cream consumption and violent crime: when temperatures rise, more people buy frozen treats and more people are out on the street where altercations occur. Once we account for temperature, the association between ice cream and crime largely disappears. Ice cream and crime move together not because one causes the other, but because both respond to a third variable.

Political data are full of similar puzzles, only the lurking variables are harder to spot. States with higher education spending tend to have better student outcomes, but wealthy states both spend more on schools and have many other advantages. Countries that adopt democracy often grow faster than countries that do not, but the countries selecting into democracy may differ in dozens of unobserved ways. Voters who watch the news are more politically knowledgeable, but the same people who choose to follow politics are the people who would have been knowledgeable regardless. In every case, correlation alone cannot deliver a verdict.

2.3 The Three Requirements for Causality

The standard framework for evaluating causal claims has three requirements. To claim that a treatment X causes an outcome Y, you must establish all three. If any one fails, the causal claim falls.

Requirement	Question	Difficulty
Association	Are X and Y related in the data?	Usually easy
Temporal ordering	Does X occur before Y?	Sometimes hard
No confounding	Could a Z cause both X and Y?	Almost always hard

Table 2: The three requirements for a causal claim.

The three requirements work together. Association is necessary but easily achieved. Temporal ordering rules out the most basic form of reverse causation. The absence of confounding is the hardest to establish and is the central concern of the rest of this chapter.

2.3.1 Requirement 1: Association

Association means that when X takes different values, Y takes systematically different values too. We saw in earlier chapters how to test for association: comparing group means, calculating correlations, building cross-tabulations, plotting scatterplots. Each tool answers the same question in a different form. Do treatment and control units tend to look different on the outcome?

Consider a simulated example of state social spending and poverty rates.

```
cor(policy_data$social_spending, policy_data$poverty_rate)
```

```
[1] -0.6890609
```

> **What this code does:** We compute the correlation between simulated per-capita social spending and the poverty rate across 50 states. The correlation is strongly negative: states that spend more on social programs have lower poverty rates. That association is necessary for any claim that spending reduces poverty, but it is far from sufficient. We have not ruled out reverse causation (low-poverty states can afford more spending) or confounding (wealthy states have both more spending and lower poverty for reasons unrelated to the programs themselves).

2.3.2 Requirement 2: Temporal Ordering

The cause must come before the effect. This seems trivially obvious, but it is surprisingly easy to violate in political data. Most surveys are *cross-sectional*, meaning every variable is measured at the same point in time. When everything is measured at once, "before" and "after" become impossible to determine from the data alone.

Consider the relationship between political knowledge and turnout. Knowledgeable citizens vote at higher rates, but does knowledge cause voting, or does voting cause knowledge? Both directions are plausible. People who plan to vote may invest in learning about the candidates. People

who already follow politics may be more inclined to show up. A single cross-sectional snapshot cannot distinguish these stories. Researchers address this with panel data (the same individuals interviewed across multiple waves), natural timing (events with a clear before-and-after structure), or experimental designs that fix the order of treatment and outcome by construction.

Even temporal precedence is not enough on its own. Roosters crow before sunrise, but they do not cause the sun to rise. Temporal ordering is necessary, not sufficient.

2.3.3 Requirement 3: No Confounding

A confounder is a third variable Z that affects both the treatment X and the outcome Y. Confounders create associations that look causal but are not. They are the deepest threat to causal inference because there are potentially infinitely many of them, and we can never prove we have ruled them all out.

Consider the relationship between education and income. People with more years of schooling earn more on average. But the kind of person who pursues additional education is, on average, different from the kind of person who does not. They may come from wealthier families, have stronger cognitive skills, be more motivated, or live in regions with better job markets. Any of these factors can independently raise both education and income. Without somehow holding them constant, the simple comparison conflates the effect of schooling with the effects of every other characteristic that differs across these groups.

> **A Confounder Checklist**
>
> Whenever you encounter an apparent causal claim, walk through these questions before accepting it:
>
> **1. Could a third variable cause both?** Is there a Z that plausibly influences both the alleged cause and the alleged effect?
>
> **2. Are the units self-selected?** Did the units that received the treatment choose to receive it? If so, they probably differ from those that did not in ways that matter for the outcome.
>
> **3. Could causation run the other way?** Could the outcome be influencing the treatment instead of the reverse?
>
> **4. Could both be responding to a trend?** Are both variables changing over time for reasons unrelated to each other?
>
> If any answer is ”yes” and the analysis has not addressed it, the causal claim is shaky.

2.4 Spurious Correlations

A spurious correlation is an association between two variables that is not due to a causal relationship between them. Spurious correlations are unavoidable in large data sets. Tyler Vigen’s

well-known collection includes a near-perfect correlation between U.S. per-capita margarine consumption and the divorce rate in Maine, and another between the number of films Nicolas Cage appears in each year and the number of swimming pool drownings. Nobody believes margarine ends marriages or that Mr. Cage's filmography drowns people. The variables happen to trend together over time, and trending variables will appear correlated whether or not they are causally related.

The political science version of this problem is more dangerous because the spurious correlations are less obviously absurd. Countries with more McDonald's restaurants rarely fight wars with each other, but this is not because Big Macs prevent conflict. It is because countries wealthy and stable enough to host major fast-food chains are also countries with the institutional and economic incentives to avoid interstate war. Cities with more coffee shops have higher voter turnout, but this is not because espresso boosts civic engagement. Both coffee-shop density and turnout reflect demographic features of educated, professional neighborhoods.

> **When to suspect a spurious correlation**
>
> - The two variables both trend over time (anything growing or shrinking).
> - The mechanism connecting cause and effect is vague or implausible.
> - The relationship would not survive a careful comparison of similar units.
> - The author has not discussed alternative explanations.

2.5 Confounding Variables in Political Science

Confounding is so common in political data that learning to spot it is one of the most useful skills you will develop in this course. Let us walk through several political examples and identify the confounder in each.

Campaign spending and vote share. Candidates who raise more money win more votes. Does spending cause votes? Probably some, but the strong raw correlation overstates the effect, because high-quality candidates both raise more money (donors back winners) and earn more votes (voters prefer the better candidate). Candidate quality is the confounder.

Media exposure and political knowledge. Citizens who consume more news are more knowledgeable. Does the news teach them, or do already-knowledgeable citizens choose to follow news more closely? Pre-existing political interest plausibly drives both. Interest is the confounder.

Refugee contact and attitudes toward integration. In some studies, native-born residents who live near refugees report more positive views of refugee integration. Does contact cause warmer attitudes, or do warmer people move into more diverse neighborhoods? Self-selection into neighborhoods is the confounder; a careful study has to break the link between residence and attitude.

Negative advertising and electoral victory. Campaigns that go negative often win. Does nega-

tivity work, or do desperate trailing campaigns disproportionately go negative? Initial position in the race is the confounder, and it points in the opposite causal direction from the naive interpretation.

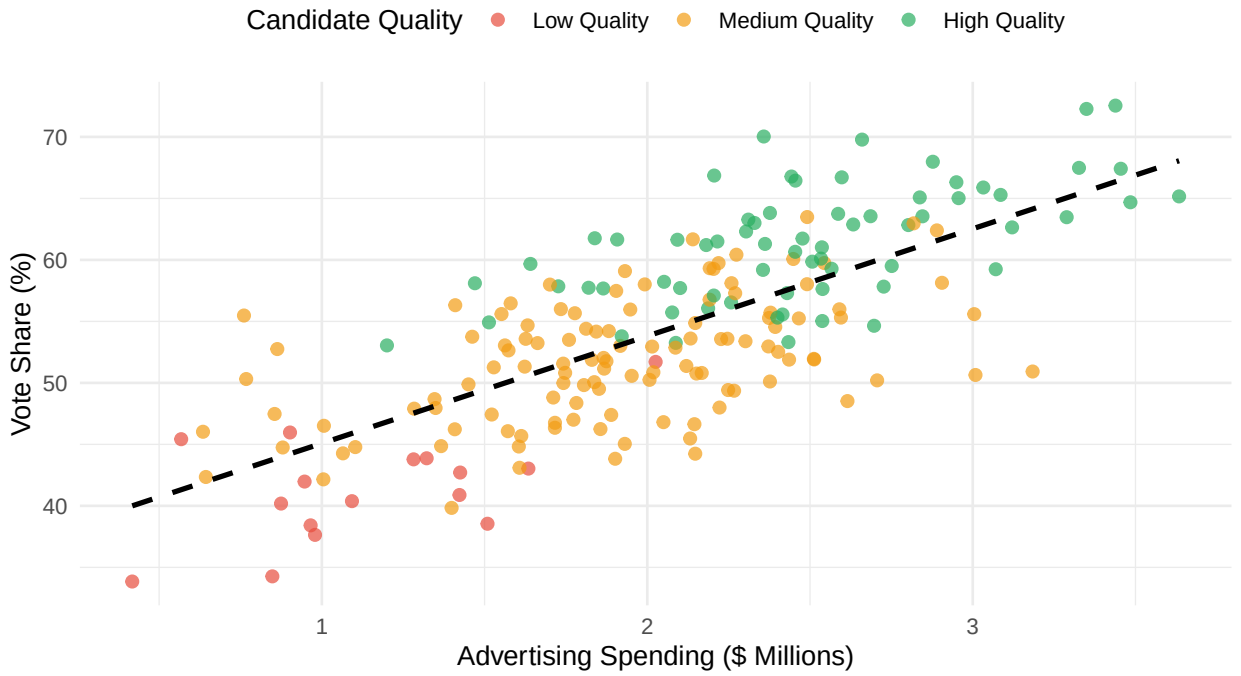

Figure 1: Figure 4.1: A confounded relationship. Campaign spending and vote share are correlated, but most of the association reflects candidate quality, which drives both spending and votes.

```
naive_model <- lm(votes ~ spending, data = ovb_data)
adjusted_model <- lm(votes ~ spending + quality, data = ovb_data)

cat("Naive estimate (no control):",
    round(coef(naive_model)["spending"], 2), "\n")
```

```
Naive estimate (no control): 8.72
```

```
cat("Adjusted estimate (with quality):",
    round(coef(adjusted_model)["spending"], 2), "\n")
```

```
Adjusted estimate (with quality): 2.64
```

```
cat("True effect built into simulation:", 2, "\n")
```

```
True effect built into simulation: 2
```

> **What this code does:** We simulate a world where the true effect of one extra million dollars of campaign spending is two percentage points of vote share. Without controlling for candidate quality, the naive estimate is far too large, because high-quality candidates both raise more money and win more votes. Controlling for quality recovers something much closer to the true effect. The lesson is sobering: the same data, analyzed without thinking about confounders, can tell you that the effect of money is roughly four times larger than it really is.

2.6 Spotting Confounders in the Wild

Identifying potential confounders is partly a craft. The questions to ask are always the same, but the answers depend on what you know about the substantive setting. Why might these units have received the treatment? What else changed at the same time? Who is in this sample and how did they get there? In each case, the goal is to imagine a Z that pushes both the treatment and the outcome and that could explain the pattern in the data without invoking a causal link.

The hardest confounders are the unobserved ones: variables that affect both treatment and outcome but that are not in our data set. We can adjust for variables we measure, but unmeasured confounders remain a permanent threat. This is why the gold standard for causal inference is to design the study so that confounders cannot drive the result, rather than to control for them after the fact.

2.7 Using AI to Identify Alternative Explanations

Modern AI tools can play a productive role in this kind of brainstorming. Large language models are particularly good at generating long lists of plausible confounders, alternative mechanisms, and reverse-causation stories. They will not tell you which explanation is correct, but they will help you avoid the dangerous mistake of accepting the first plausible story you encounter.

> **Example Prompt for Confounder Brainstorming**
>
> *"I have found that cities with more public libraries also have higher voter turnout in local elections. Before I conclude that libraries cause civic engagement, please brainstorm a list of variables that might independently cause both library density and voter turnout. For each, briefly explain the mechanism."*
>
> A well-crafted prompt does three things:
>
> 1. It states the finding precisely.
> 2. It asks for alternatives, not validation.
> 3. It requests reasoning, not just a list, so you can evaluate each candidate.

> **Example Prompt for Causal Critique**
>
> *"Evaluate the following causal claim against the three requirements for causality (association, temporal ordering, no confounding): 'Heavy social media users score lower on political knowledge tests, so social media reduces political knowledge.' For each requirement, explain whether the study described meets it, and identify the strongest alternative explanation."*
>
> This kind of structured critique forces the AI to apply the framework we just developed rather than offering vague commentary. Your job is to evaluate the AI's reasoning, not to defer to it.

A note on AI limitations

AI models will happily produce confident-sounding causal arguments that are wrong. They are useful as a brainstorming partner, not as an authority. Treat suggested confounders as hypotheses to verify, not as conclusions. Always demand mechanisms; ”demographics” is not a confounder until you can say which demographic and why it matters.

2.8 Common Mistakes in Causal Reasoning

Before we move to the modern causal-inference toolkit, it is worth cataloguing the most common errors in causal reasoning. Each is a way of failing one of the three requirements.

Mistake 1: Post hoc ergo propter hoc

Latin for ”after this, therefore because of this.” The error is assuming that because B followed A, A must have caused B. The economy improved after a new administration took office, so the administration must have caused the improvement. Crime fell after a new police chief was appointed, so the chief must be responsible. Many things follow many other things without any causal connection.

Mistake 2: Reverse causation

Getting the direction of causation backwards. Do good policies cause economic growth, or does economic growth give politicians the slack to enact good policies? Do happy people make more money, or does making more money make people happier? Reverse causation is most dangerous when both directions are individually plausible.

Mistake 3: Selection bias

When the units that received treatment differ systematically from those that did not, and the differences themselves affect the outcome. Volunteers for a civic-engagement program differ from non-volunteers; private-school students differ from public-school students; refugees who settle in a particular city differ from those who settle elsewhere. Any comparison that ignores the selection process confounds the effect of the program with the effect of who signed up.

Mistake 4: Selecting on the outcome

Studying only successful cases and then drawing conclusions about what caused success. If you interview only billionaires and find they all read books, you have not learned that reading creates wealth; you have only learned that billionaires read. You need to compare successes to failures. The same logic applies to political movements, election victories, and policy reforms.

2.9 Bridging to Segment B

Segment A has been mostly philosophical: when does it make sense to say that X causes Y, and what can go wrong when we claim it does? The three requirements (association, temporal ordering, no confounding) are necessary conditions, but they do not tell us how to satisfy them with real data. The rest of this chapter takes up that practical question. The fundamental problem of causal inference makes the third requirement painfully concrete. Randomized experiments offer a clean (when feasible) solution. Difference-in-differences provides a powerful alternative when experiments are off the table.

The thread connecting both segments is the counterfactual. To say that policy X caused outcome Y is to say that without X, Y would have been different. The philosophical requirements ensure we are asking the right question. The modern toolkit gives us credible ways to estimate the difference.

3 Using AI for This Material

Where AI Helps This Week

- Writing long `case_when()` ladders with many branches without dropping a condition or mismatching types.
- Suggesting natural cutpoints (quartiles, conventional age brackets, income deciles) when converting a continuous variable to categories.
- Explaining what `if_else()` returns when the two branches have different types and why R complains.

Where AI Gets This Wrong This Week

- Choosing between an experimental and an observational design for a specific research question depends on what you can manipulate, what is ethical, and what data exist — judgments AI cannot make for you.
- It will not flag when a chain of `mutate()` calls is implicitly p-hacking by searching for

a specification that "works."

- It will turn a design question into a code request and produce a regression that looks causal even when the comparison is not valid.

A prompt that works for this week:

> Write a `case_when()` that recodes `age` (a numeric column in years) into a factor called `age_group` with levels `Under 30`, `30 to 44`, `45 to 64`, and `65 and over`.

A prompt that misleads:

> I have data on districts that adopted voter ID laws and districts that did not. Write me the code to estimate the causal effect of voter ID on turnout.

Why it misleads: the request collapses a design question (are the groups comparable? is there a parallel-trends assumption?) into a code request, and AI will happily produce a regression that looks causal but is not.

Where AI Helps This Week

- Explaining the difference between association and causation in a worked example, including spelling out the counterfactual a study is implicitly comparing against.
- Defining DAGs, confounders, mediators, and colliders in the abstract and walking through textbook examples of each.
- Articulating what the SUTVA (no-interference) assumption requires and where it tends to fail.

Where AI Gets This Wrong This Week

- Identifying confounders for a specific empirical setting requires substantive knowledge of the political context, which AI does not have.
- Evaluating whether the parallel-trends assumption holds in your data requires looking at pre-treatment trends, which AI cannot inspect.
- It often fails to recognize post-treatment bias when you describe a "control" variable that is really an outcome of treatment.

A prompt that works for this week:

> I am studying whether televised debates change voter preferences. List the most likely confounders that would make a simple correlation between debate-watching and vote choice misleading, and explain why each one matters.

A prompt that misleads:

> Here is my dataset on minimum-wage increases and county employment. Did the policy cause the change?

Why it misleads: causal identification requires evaluating an assumption (parallel trends, ignorability, exclusion) against the specific data-generating process, which AI cannot inspect — it will return a generic difference-in-differences answer without asking whether the design is credible.

4 Common Mistakes and How to Avoid Them

Mistake 1: Using summarise() When You Meant mutate()

`summarise()` collapses a dataset to one row per group. `mutate()` adds a column without changing the number of rows. Students often write `summarise(age_group = case_when(...))` and watch every other column disappear. If you want each respondent to receive a new value, the verb is almost always `mutate()`.

Mistake 2: Forgetting the TRUE Catch-All in case_when()

If none of your conditions in `case_when()` match a row, that row gets `NA`. Always end with `TRUE ~ "Other"` or a similar safety net, unless you genuinely want missing values for unmatched cases. This is the single most common silent bug in tidyverse code.

Mistake 3: Treating Observational Patterns as Causal

A correlation between two variables in an observational dataset is consistent with many causal stories, including the story that some third factor produced both. No amount of variable creation can fix this. `mutate()` cannot rescue an observational comparison from confounding. Only a credible research design can.

Mistake 4: Overinterpreting Natural Experiments

Natural experiments live or die by the as-if-random assumption. Before trusting one, ask: could the units have anticipated the assignment? Could they have manipulated it? Are there other channels through which the assignment could affect the outcome? A lottery is only as clean as its implementation, and many designs that look quasi-random under one assumption look quite different under another.

5 R Functions Reference

Function	Purpose	Example
`mutate()`	Create or modify columns	`mutate(margin = dem - rep)`
`if_else()`	Two-way recoding	`if_else(age >= 65, "Senior", "Not")`
`case_when()`	Multi-way recoding	`case_when(age < 30 ~ "Young", ...)`
`pull()`	Extract a column as a vector	`pull(med)`
`n_distinct()`	Count unique values	`n_distinct(state)`
`across()`	Apply over multiple columns	`across(ends_with("vote"), ...)`
`recode()`	Swap specific values	`recode(x, "old" = "new")`
`count()`	Tally rows per group	`count(treatment, voted)`

Table 3: Key R Functions for This Class

Function	Purpose	Example
`cor()`	Correlation between two variables	`cor(x, y)`
`lm()`	Fit a linear model	`lm(y ~ treat, data)`
`tidy()`	Tidy regression output	`tidy(model)`
`group_by()`	Group rows for summarization	`group_by(treatment)`
`summarise()`	Collapse groups to summaries	`summarise(mean(y))`
`pivot_wider()`	Reshape long data to wide	`pivot_wider(...)`

Table 4: Key R functions used in Chapter 4.

6 Practice Problems

Question 1

You have a dataset of congressional districts with columns `dem_votes`, `rep_votes`, and `incumbent_party`. Using `mutate()` and `case_when()`, create a variable `competitiveness` that classifies each district as "Safe Dem," "Lean Dem," "Toss-Up," "Lean Rep," or "Safe Rep" based on the Democratic vote share. Explain the cutpoints you chose and what theoretical assumptions they encode.

Question 2

A campaign claims that voters who received its mailers turned out at a higher rate than voters who did not. Why is this comparison unconvincing as evidence that the mailers worked? Describe two confounders that could produce the same pattern. What design would the

campaign need to use to make a credible causal claim?

Question 3

Compare a cross-sectional survey of American voters in 2024 to a panel survey that interviewed the same voters in 2020, 2022, and 2024. What question can the panel answer that the cross-section cannot? What practical disadvantages does the panel design carry?

Question 4 (Computational)

Create a simulated dataset of 500 voters with `age`, `party`, and a randomly assigned `treatment` indicator. Use `mutate()` and `case_when()` to create an `age_group` variable with four categories. Use `count()` to verify that the treatment and control groups have similar age-group distributions. What would you conclude if they did not?

For Further Study

Wickham and Grolemund's *R for Data Science* provides an extended treatment of the tidyverse transformation verbs introduced here, with many more worked examples. For research design, Angrist and Pischke's *Mostly Harmless Econometrics* remains the most accessible introduction to modern causal inference, while Dunning's *Natural Experiments in the Social Sciences* catalogs the most influential as-if-random designs in political science. As you read these works, notice how often the analytical move that makes a study compelling is a small, careful transformation of a variable, combined with a research design that gives that transformation meaning.

Question 1: Confounder Hunt

A study reports that congressional districts represented by women have lower defense spending requests. The author concludes that female representatives cause lower defense spending. Identify at least two plausible confounders that could generate this association without a causal effect of representative gender. For each, briefly describe the mechanism.

Question 2: Designing a GOTV Experiment

A city wants to know whether a new text-message GOTV program increases turnout. Sketch a randomized experiment to evaluate the program. Specify the population, the unit of randomization, the treatment, the control condition, and the outcome. What is the ATE you would estimate, and how would you compute it?

Question 3: A DiD Calculation

A neighboring-states study of minimum wage finds that fast-food employment in the treatment

state rose from 21.0 to 22.3, while employment in the comparison state fell from 22.5 to 21.1. Compute the DiD estimate and explain what it implies about the effect of the minimum-wage increase. What assumption must hold for this estimate to be credible?

Question 4 (Computational)

Simulate a randomized experiment with 800 units. Randomly assign half to treatment. Generate an outcome equal to 50 plus 4 times the treatment indicator plus normally distributed noise with standard deviation 8. Compute the ATE by (a) taking the difference in group means and (b) running a linear regression of outcome on treatment. Verify that the two approaches give the same point estimate, and report the regression's standard error on the treatment coefficient.

Question 5: Critique a Media-Study Claim

A widely shared op-ed argues that "people who watch cable news are more politically polarized, so cable news is polarizing the country." Apply the three requirements from Segment A and the potential-outcomes framework from Segment B to critique this claim. What design would you propose to estimate the causal effect of cable-news exposure on polarization?

For Further Study

Students interested in going deeper should look at Joshua Angrist and Jorn-Steffen Pischke's *Mastering 'Metrics*, which offers an accessible treatment of randomized experiments and difference-in-differences with detailed empirical examples. Judea Pearl's *The Book of Why* provides a complementary philosophical perspective on causation and the limits of correlation-based reasoning. For political-science applications specifically, Don Green and Alan Gerber's *Get Out the Vote* summarizes a remarkable program of GOTV field experiments and demonstrates what credible causal inference looks like in practice.